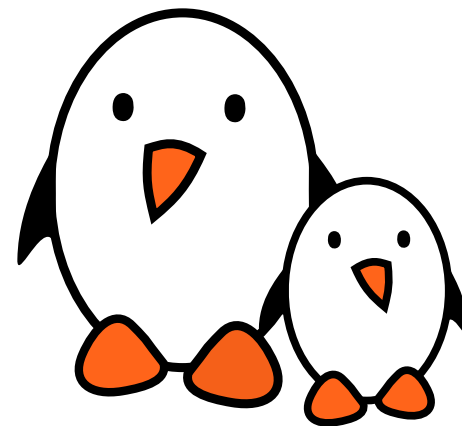




Userspace ALSA

bootlin





alsa-lib



- ▶ The main way to interact with ALSA devices is to use alsa-lib.
- ▶ <https://github.com/alsa-project/alsa-lib>
- ▶ It provides mainly access to the devices but also goes further and allows handling audio in userspace.
- ▶ The library itself is actually named `libasound`
- ▶ The include file is `alsa/asoundlib.h`



alsa-lib API

▶ `int snd_pcm_open(snd_pcm_t ** pcm, const char * name, snd_pcm_stream_t stream, int mode)`

▶ `name` is the name of the PCM to be opened.

▶ `stream` can be either `SND_PCM_STREAM_PLAYBACK` or

▶ `mode` can be a combination of `SND_PCM_NONBLOCK` and

▶ `int snd_pcm_close(snd_pcm_t *pcm)`

▶ Closes the PCM.



PCM name

- ▶ This can be specified as a hardware device. The three arguments (in order: CARD,DEV,SUBDEV) specify card number or identifier, device number and subdevice number (-1 means any). For example: `hw:0` or `hw:1,0`. Instead of the index, the card name can be used: `:STM32MP15DK,0`
- ▶ Or through the `plug` plugin: `plug:mypcmdef`, `plug:hw:0,0`.
- ▶ The list of available names can be generated using `snd_card_next` to iterate over all the physical cards. See `device_list` in `aplay`.



- ▶ The next step is to handle the PCM stream parameters
- ▶

```
snd_pcm_hw_params_t *hw_params; int snd_pcm_hw_params_malloc(snd_pcm_hw_params_t ** ptr)  
int snd_pcm_hw_params_any(snd_pcm_t * pcm, snd_pcm_hw_params_t * params)
```
- ▶ This will allocate a `snd_pcm_hw_params_t` and fill it with the range of parameters supported by `pcm`.
- ▶

```
int snd_pcm_hw_params_set_access(snd_pcm_t *pcm, snd_pcm_hw_params_t *params,  
                                snd_pcm_access_t _access)
```
- ▶ This set the proper access type: interleaved or non-interleaved, mmap or not.
- ▶

```
int snd_pcm_hw_params_set_format(snd_pcm_t *pcm, snd_pcm_hw_params_t *params,  
                                snd_pcm_format_t val)
```
- ▶ This set the format, using a `SND_PCM_FORMAT_` macro.



- ▶ `int snd_pcm_hw_params_set_channels(snd_pcm_t *pcm, snd_pcm_hw_params_t *params, unsigned int val)`
- ▶ Sets the number of channels.
- ▶ `int snd_pcm_hw_params_set_rate_near(snd_pcm_t *pcm, snd_pcm_hw_params_t *params, unsigned int *val, int *dir)`
- ▶ Sets the sample rate, setting `dir` to 0 will require the exact rate.
- ▶ `int snd_pcm_hw_params_set_periods(snd_pcm_t *pcm, snd_pcm_hw_params_t *params, unsigned int val, int dir)`
`int snd_pcm_hw_params_set_period_size(snd_pcm_t *pcm, snd_pcm_hw_params_t *params, snd_pcm_uframes_t val, int dir)`
`int snd_pcm_hw_params_set_buffer_size(snd_pcm_t *pcm, snd_pcm_hw_params_t *params, snd_pcm_uframes_t val)`
- ▶ Sets the number of periods and the period size in the buffer or the buffer size.



alsa-lib API - PCM

- ▶ `int snd_pcm_hw_params(snd_pcm_t * pcm, snd_pcm_hw_params_t * params)`
- ▶ Installs the parameters and calls `snd_pcm_prepare` on the stream.
- ▶ `void snd_pcm_hw_params_free(snd_pcm_hw_params_t * obj)`
- ▶ Frees the allocated `snd_pcm_hw_params_t`.
- ▶ `int snd_pcm_prepare(snd_pcm_t * pcm)`
- ▶ Prepares the stream.
- ▶ `int snd_pcm_wait(snd_pcm_t * pcm, int timeout)`
- ▶ Waits for the PCM to be ready.



alsa-lib API - PCM

- ▶ `snd_pcm_sframes_t snd_pcm_writei(snd_pcm_t *pcm, const void *buffer, snd_pcm_uframes_t size)`
`snd_pcm_sframes_t snd_pcm_readi(snd_pcm_t *pcm, void *buffer, snd_pcm_uframes_t size)`
`snd_pcm_sframes_t snd_pcm_writen(snd_pcm_t *pcm, void **bufs, snd_pcm_uframes_t size)`
`snd_pcm_sframes_t snd_pcm_readn(snd_pcm_t *pcm, void **bufs, snd_pcm_uframes_t size)`

- ▶ Write or read from an interleaved or non-interleaved buffer.

- ▶ `int snd_pcm_mmap_begin(snd_pcm_t *pcm, const snd_pcm_channel_area_t **areas, snd_pcm_uframes_t *offset, snd_pcm_uframes_t *frames)`
`snd_pcm_sframes_t snd_pcm_mmap_commit(snd_pcm_t *pcm, snd_pcm_uframes_t offset, snd_pcm_uframes_t frames)`
`snd_pcm_sframes_t snd_pcm_mmap_writei(snd_pcm_t *pcm, const void *buffer, snd_pcm_uframes_t size)`
`snd_pcm_sframes_t snd_pcm_mmap_readi(snd_pcm_t *pcm, void *buffer, snd_pcm_uframes_t size)`
`snd_pcm_sframes_t snd_pcm_mmap_writen(snd_pcm_t *pcm, void **bufs, snd_pcm_uframes_t size)`
`snd_pcm_sframes_t snd_pcm_mmap_readn(snd_pcm_t *pcm, void **bufs, snd_pcm_uframes_t size)`

- ▶ Write or read from an interleaved or non-interleaved mmap buffer.



alsa-lib API - controls

- ▶ It is possible to set controls programatically.

- ▶ `snd_ctl_t *handle; int snd_ctl_open (snd_ctl_t **ctl, const char *name, int mode)`

- ▶ Opens the sound card to be controlled.

- ▶

```
snd_ctl_elem_id_t *id;
#define snd_ctl_elem_id_alloca(ptr)
snd_ctl_elem_value_t *value;
#define snd_ctl_elem_value_alloca(ptr)
```

- ▶ Allocate a `snd_ctl_elem_id_t` , referring to a particular control and a `snd_ctl_elem_value_t` to be set for this control.

- ▶

```
void snd_ctl_elem_id_set_interface(snd_ctl_elem_id_t *obj, snd_ctl_elem_iface_t val)
void snd_ctl_elem_id_set_name(snd_ctl_elem_id_t *obj, const char *val)
```

- ▶ Set the interface and name of the control to be set.



- ▶ A lookup is needed to fill the `snd_ctl_elem_id_t` completely

```
int lookup_id(snd_ctl_elem_id_t *id, snd_ctl_t *handle)
{
    int err;
    snd_ctl_elem_info_t *info;
    snd_ctl_elem_info_alloca(&info);

    snd_ctl_elem_info_set_id(info, id);
    if ((err = snd_ctl_elem_info(handle, info)) < 0) {
        return err;
    }
    snd_ctl_elem_info_get_id(info, id);

    return 0;
}
```



alsa-lib API - controls

- ▶ `void snd_ctl_elem_value_set_id(snd_ctl_elem_value_t *obj, const snd_ctl_elem_id_t *ptr)`
- ▶ Links the value with the control id.
- ▶
 - `void snd_ctl_elem_value_set_boolean(snd_ctl_elem_value_t *obj, unsigned int idx, long val)`
 - `void snd_ctl_elem_value_set_integer(snd_ctl_elem_value_t *obj, unsigned int idx, long val)`
 - `void snd_ctl_elem_value_set_integer64(snd_ctl_elem_value_t *obj, unsigned int idx, long long val)`
 - `void snd_ctl_elem_value_set_enumerated(snd_ctl_elem_value_t *obj, unsigned int idx, unsigned int val)`
 - `void snd_ctl_elem_value_set_byte(snd_ctl_elem_value_t *obj, unsigned int idx, unsigned char val)`
 - `void snd_ctl_elem_set_bytes(snd_ctl_elem_value_t *obj, void *data, size_t size)`
- ▶ Set the value in `snd_ctl_elem_value_t`.
- ▶ `int snd_ctl_elem_write(snd_ctl_t *ctl, snd_ctl_elem_value_t *data)`
- ▶ Actually set the control.

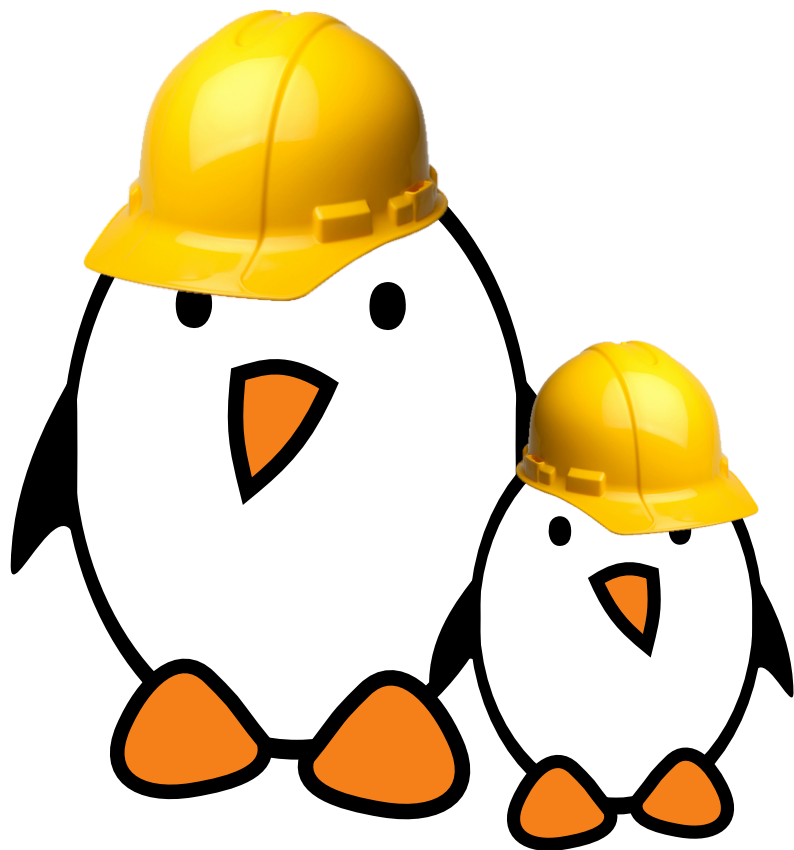


Going further

- ▶ UCM: The ALSA Use Case Configuration: https://www.alsa-project.org/alsa-doc/alsa-lib/group__ucm__conf.html
- ▶ ALSA topology: https://www.alsa-project.org/wiki/ALSA_topology



Demo - Card configuration examples



Using `alsa-lib` tools to:

- ▶ Reorder channels
- ▶ Split channels
- ▶ Resample
- ▶ Mix samples
- ▶ Apply effects